

2520090008_Sathish lumar

Skill – 6

Task -1: To Store Commands, Maintain History Buffer, Handle Capacity Limits, Retrieve Entries, Display History, Validate Consistency

Creating file:

```
root@LAPTOP-4H877K4U:~# cd ~/practical_task
root@LAPTOP-4H877K4U:~/practical_task# mkdir -p ~/practical_task
cd ~/practical_task
root@LAPTOP-4H877K4U:~/practical_task# nano task1_history.c
```

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define HISTORY_CAPACITY 5
#define COMMAND_SIZE 256

/* History buffer */
char history[HISTORY_CAPACITY][COMMAND_SIZE];

int count = 0;
int start = 0;
```

```

/* =====

Add command to history

===== */

void add_command(const char *command)
{
    int position;

    if (count < HISTORY_CAPACITY)
    {
        /*
         * Buffer is not full.
         * Add command at the next available position.
         */
        position = (start + count) % HISTORY_CAPACITY;

        strcpy(history[position], command);

        count++;
    }
    else
    {
        /*
         * Buffer is full.
         * Replace the oldest command.
         */
        position = start;
    }
}

```

```

        strcpy(history[position], command);

        start = (start + 1) % HISTORY_CAPACITY;
    }

    printf("Command stored successfully.\n");
}

/* =====
Display history
===== */
void display_history()
{
    if (count == 0)
    {
        printf("History is empty.\n");
        return;
    }

    printf("\n===== COMMAND HISTORY =====\n");

    for (int i = 0; i < count; i++)
    {
        int position = (start + i) % HISTORY_CAPACITY;

```

```

        printf("%d %s\n", i + 1, history[position]);
    }

    printf("=====\n");
}

/* =====
Retrieve command
===== */

void get_command(int number)
{
    if (number < 1 || number > count)
    {
        printf("Invalid history number.\n");
        return;
    }

    int position = (start + number - 1) % HISTORY_CAPACITY;

    printf("Command %d: %s\n",
        number,
        history[position]);
}

/* =====
Clear history

```

```

===== */
void clear_history()
{
    count = 0;
    start = 0;

    printf("History cleared successfully.\n");
}

/* =====
   Validate history consistency
===== */
void validate_history()
{
    if (count < 0 || count > HISTORY_CAPACITY)
    {
        printf("History consistency check: FAILED\n");
        return;
    }

    if (start < 0 || start >= HISTORY_CAPACITY)
    {
        printf("History consistency check: FAILED\n");
        return;
    }
}

```

```

for (int i = 0; i < count; i++)
{
    int position = (start + i) % HISTORY_CAPACITY;

    if (strlen(history[position]) == 0)
    {
        printf("History consistency check: FAILED\n");
        return;
    }
}

printf("History consistency check: PASSED\n");
}

/* =====

Main Function

===== */

int main()
{
    char input[COMMAND_SIZE];

    printf("=====\n");
    printf("    COMMAND HISTORY BUFFER\n");
    printf("=====\n");

    printf("History capacity: %d commands\n", HISTORY_CAPACITY);

```

```
while (1)
{
    printf("\nhistoryShell> ");
    fflush(stdout);

    if (fgets(input, sizeof(input), stdin) == NULL)
    {
        break;
    }

    /* Remove newline */
    input[strcspn(input, "\n")] = '\0';

    /* Ignore empty input */
    if (strlen(input) == 0)
    {
        continue;
    }

    /* Exit */
    if (strcmp(input, "exit") == 0)
    {
        printf("Exiting history shell...\n");
        break;
    }
}
```

```
/* Display history */
if (strcmp(input, "history") == 0)
{
    display_history();
}

/* Clear history */
else if (strcmp(input, "clear") == 0)
{
    clear_history();
}

/* Validate history */
else if (strcmp(input, "validate") == 0)
{
    validate_history();
}

/* Add command */
else if (strncmp(input, "add ", 4) == 0)
{
    char *command = input + 4;

    if (strlen(command) == 0)
    {
```

```
        printf("Cannot store an empty command.\n");
    }
    else
    {
        add_command(command);
    }
}

/* Retrieve command */
else if (strncmp(input, "get ", 4) == 0)
{
    int number;

    if (sscanf(input + 4, "%d", &number) == 1)
    {
        get_command(number);
    }
    else
    {
        printf("Usage: get <number>\n");
    }
}

/* Invalid command */
else
{

```

```

        printf("Invalid command.\n");

        printf("Use: add, history, get, clear, validate, exit\n");

    }

}

return 0;

}

```

Output:

```

root@LAPTOP-4H877K4U:~# cd ~/practical_task
root@LAPTOP-4H877K4U:~/practical_task# mkdir -p ~/practical_task
cd ~/practical_task
root@LAPTOP-4H877K4U:~/practical_task# nano task1_history.c
root@LAPTOP-4H877K4U:~/practical_task# gcc task1_history.c -o task1_history
root@LAPTOP-4H877K4U:~/practical_task# ls
client      fifo_reader  fifo_writer  server      server.c    signals.c   task1_cd.c   task1_history.c  task2_builtin.c
client.c    fifo_reader.c  fifo_writer.c  server.c    signals     task1_cd   task1_history  task2_builtin
root@LAPTOP-4H877K4U:~/practical_task# ./task1_history
=====
COMMAND HISTORY BUFFER
=====
History capacity: 5 commands

historyShell> add ls
Command stored successfully.

historyShell> add pwd
Command stored successfully.

historyShell> add whoami
Command stored successfully.

```

TASK _ 2:To Create Pipeline Structures, Store Commands, Maintain Execution Order, Connect Processes, Validate Pipeline Layout, Test Configurations

Creating file:

```

root@LAPTOP-4H877K4U:~/practical_task# nano task_pipeline.c

```

code:

```

#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <unistd.h>

#include <sys/wait.h>

```

```

#define INPUT_SIZE 512

#define MAX_COMMANDS 10

#define MAX_ARGS 20

typedef struct
{
    char *args[MAX_ARGS];
} Command;

/* Remove leading and trailing spaces */
char *trim_spaces(char *str)
{
    while (*str == ' ')
    {
        str++;
    }

    char *end = str + strlen(str) - 1;

    while (end >= str && *end == ' ')
    {
        *end = '\0';
        end--;
    }

    return str;
}

/* Parse one command into arguments */
void parse_arguments(char *command, Command *cmd)
{
    int index = 0;

    char *token = strtok(command, " ");

```

```
while (token != NULL && index < MAX_ARGS - 1)
{
    cmd->args[index] = token;
    index++;

    token = strtok(NULL, " ");
}

cmd->args[index] = NULL;
}

/* Validate pipeline */
int validate_pipeline(char *input)
{
    int length = strlen(input);

    if (length == 0)
    {
        printf("Error: Empty pipeline.\n");
        return 0;
    }

    if (input[0] == '|')
    {
        printf("Error: Pipeline cannot start with '|'.\n");
        return 0;
    }

    if (input[length - 1] == '|')
    {
        printf("Error: Pipeline cannot end with '|'.\n");
        return 0;
    }

    for (int i = 0; i < length - 1; i++)
```

```

{
    if (input[i] == '|' && input[i + 1] == '|')
    {
        printf("Error: Empty command between pipes.\n");
        return 0;
    }
}

return 1;
}

/* Execute pipeline */
void execute_pipeline(Command commands[], int command_count)
{
    int pipefds[2 * (command_count - 1)];

    for (int i = 0; i < command_count - 1; i++)
    {
        if (pipe(pipefds + i * 2) == -1)
        {
            perror("pipe");
            exit(1);
        }
    }

    int process_index = 0;

    for (int i = 0; i < command_count; i++)
    {
        pid_t pid = fork();

        if (pid == -1)
        {
            perror("fork");
            exit(1);

```

```

}

if (pid == 0)
{
    /*
     * If this is not the first command,
     * connect previous pipe read end to stdin.
     */
    if (i > 0)
    {
        if (dup2(pipefds[(i - 1) * 2], STDIN_FILENO) == -1)
        {
            perror("dup2 input");
            exit(1);
        }
    }
}

/*
 * If this is not the last command,
 * connect stdout to current pipe write end.
 */
if (i < command_count - 1)
{
    if (dup2(pipefds[i * 2 + 1], STDOUT_FILENO) == -1)
    {
        perror("dup2 output");
        exit(1);
    }
}

/* Close all pipe descriptors in child */
for (int j = 0; j < 2 * (command_count - 1); j++)
{
    close(pipefds[j]);
}

```

```

        execvp(commands[i].args[0], commands[i].args);

        perror("execvp");

        exit(1);
    }

    process_index++;
}

/* Parent closes all pipe descriptors */
for (int i = 0; i < 2 * (command_count - 1); i++)
{
    close(pipefds[i]);
}

/* Wait for all child processes */
for (int i = 0; i < process_index; i++)
{
    wait(NULL);
}
}

int main()
{
    char input[INPUT_SIZE];

    printf("=====\n");
    printf("    PIPELINE EXECUTION PROGRAM\n");
    printf("=====\n");

    printf("Example:\n");
    printf("ls | grep txt | wc -l\n");

    while (1)

```

```

{

printf("\npipelineShell> ");

fflush(stdout);

if (fgets(input, sizeof(input), stdin) == NULL)
{
    break;
}

input[strcspn(input, "\n")] = '\0';

if (strcmp(input, "exit") == 0)
{
    printf("Exiting pipeline shell...\n");
    break;
}

if (!validate_pipeline(input))
{
    continue;
}

Command commands[MAX_COMMANDS];

char *command_strings[MAX_COMMANDS];

int command_count = 0;

char *token = strtok(input, "|");

while (token != NULL &&
        command_count < MAX_COMMANDS)
{
    token = trim_spaces(token);

```

```
    if (strlen(token) == 0)
    {
        printf("Error: Empty command detected.\n");
        command_count = 0;
        break;
    }

    command_strings[command_count] = token;
    command_count++;
    token = strtok(NULL, "|");
}
if (command_count == 0)
{
    continue;
}
printf("\nPipeline structure:\n");
for (int i = 0; i < command_count; i++)
{
    printf("Command %d: %s\n",
        i + 1,
        command_strings[i]);
    parse_arguments(command_strings[i],
        &commands[i]);
}
printf("\nExecuting pipeline...\n\n");

execute_pipeline(commands,
    command_count);
}
return 0;
}
```